# Day 3 Evidence Workbooklet

Types, Casting, and Truthiness

Engineering practice → data type → safe conversion → truthiness check → support next step

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

| | | | | | |
|---|---|---|---|---|---|
| **Name** | | **Section** | | **Date** | |
| **Score** | _____/43 | | Mastery check | secure / developing / needs support | |

Your team is continuing the pump-flow record from Day 2. Today the values arrive from an intake form before they are used in a notebook. Some entries arrive as text, some are already numbers, and some fields are blank. Before the team can make safe threshold decisions tomorrow, the record has to distinguish type, conversion, and truthiness.

The problem today is not branch routing. The problem is whether a value is text, a number, or Boolean-like evidence, and whether a conversion or truthiness check is safe to trust.

| | |
|---|---|
| **Engineering task** | Prepare a pump-flow intake record so later threshold and go/no-go decisions are based on converted values rather than misleading text labels. |
| **Computational move** | Classify values by type, cast text when safe, and interpret truthiness without confusing a non-empty label for an approved result. |
| **Python focus** | `str`, `float`, `int`, `bool`, `type(...)`, `float(...)`, empty string, non-empty string, zero, and nonzero values. |
| **Evidence to keep** | original value, Python type, cast expression, cast result, truthiness result, and one corrected intake-note sentence. |

## Day 3 type and truthiness rules

1. Text that looks like a number is still text until code converts it. Example: `"18.6"` is a string, while `18.6` is a float.
2. A cast such as `float("18.6")` produces a numeric value, but it does not change the original text variable unless an assignment stores the result.
3. Empty strings are falsey. Non-empty strings are truthy, even when the text says `"no"` or `"0"`.
4. Numeric zero is falsey. Nonzero numbers are truthy.
5. Truthiness is evidence about empty/non-empty or zero/nonzero status; it is not the same as engineering approval.

## 1 Read the intake record as typed data

recognize predict

Use the intake record below. First identify what kind of value each line stores. Do not make a go/no-go decision yet.

```
# Intake form values from the pump station
pump_id = "P-22"
target_flow_text = "20.0"
measured_flow_text = "18.6"
operator_note = "needs review"
approved_text = "no"
spare_comment = ""
retry_count = 0
sensor_ready = True

# Safe conversions for later calculations
target_flow_lpm = float(target_flow_text)
measured_flow_lpm = float(measured_flow_text)
gap_lpm = target_flow_lpm - measured_flow_lpm
```

| Pts. | Prompt | My evidence |
|---|---|---|
| 1 | Which two variables store numbers as text before conversion? | |
| 1 | Which variable stores an empty string? | |
| 1 | Which variable stores a whole-number count rather than text? | |
| 1 | Which variable already stores a Boolean value? | |

## 2 Classify type before using a value

recognize explain

Complete the type table. Use Python type names such as `str`, `int`, `float`, and `bool`. Then write why the type matters for later work.

| Pts. | Variable or expression | Type | Why this matters before a threshold check |
|---|---|---|---|
| 2 | `target_flow_text` | | |
| 2 | `target_flow_lpm` | | |
| 2 | `approved_text` | | |
| 2 | `sensor_ready` | | |

**Common trap to check**

The text `"20.0"` may look like a number, but Python will treat it as text until it is converted. A reliable engineering record names both the original field and the converted field.

## 3 Cast text to numbers without rewriting the source record

predict trace

A cast produces a new value. It changes a stored variable only if the cast result is assigned to a name. Complete the table.

| Pts. | Expression or assignment | Value produced or stored | What changed? |
|---|---|---|---|
| 2 | `float(target_flow_text)` | | |
| 2 | `float(measured_flow_text)` | | |
| 2 | `target_flow_lpm = float(target_flow_text)` | | |
| 2 | `gap_lpm = target_flow_lpm - measured_flow_lpm` | | |

| Pts. | Prompt | My response |
|---|---|---|
| 2 | Why should the notebook keep the original text field and the converted numeric field separate? | |
| 2 | What would be unsafe about calculating with `target_flow_text` as if it were already a number? | |

## 4 Interpret truthiness without mistaking labels for approval

predict explain

Python truthiness can be useful, but it can also mislead an engineering record when text labels are treated like decisions. Complete the table.

| Pts. | Value or variable | Truthy or falsey? | Engineering interpretation |
|---|---|---|---|
| 1 | `spare_comment` storing `""` | | |
| 1 | `operator_note` storing `"needs review"` | | |
| 1 | `approved_text` storing `"no"` | | |
| 1 | `retry_count` storing `0` | | |
| 1 | `sensor_ready` storing `True` | | |

**Truthiness warning**

`bool("no")` is `True` because the string is not empty. That does not mean the operator approved the record.

| Pts. | Prompt | My response |
|---|---|---|
| 2 | In one sentence, explain why truthiness is not the same as engineering approval. | |

## 5 Debug a type or truthiness mistake

debug test explain

A teammate writes this intake note before tomorrow's threshold check:

### Intake note to debug

The record is approved because `approved_text` has a value, and the flow gap can be calculated directly from the two text fields.

| Pts. | Prompt | My response |
|---|---|---|
| 2 | What part of the teammate's note confuses truthiness with approval? | |
| 2 | What part of the note confuses text values with numeric values? | |
| 2 | Write one non-mutating Python expression that checks the type of `approved_text`. | |
| 2 | Rewrite the intake note so it separates text fields, converted numeric fields, and actual approval evidence. | |

### Bridge to Day 4

Tomorrow's boundary checks will use comparison operators such as `<`, `<=`, and `==`. Those checks are safer when today's type evidence is already clear: converted numbers for numeric thresholds, and explicit labels or Boolean values for decision evidence.

## 6 Mastery check and support next step

transfer explain

Use your own evidence from this workbooklet. Do not write a generic answer.

| Pts. | Prompt | My response |
|---|---|---|
| 2 | Give one example where text looked like a number but needed a cast before safe calculation. | |
| 2 | Give one example where truthiness could mislead an engineering record if the team did not read the actual value. | |

### My mastery check

Mark the strongest statement that is true after this workbooklet.

| | |
|---|---|
| ☐ | Secure: I can classify a value's type, predict a safe cast, and explain why truthiness is not the same as approval. |
| ☐ | Developing: I can identify some types or casts, but I still need support with string truthiness or converted fields. |
| ☐ | Needs support: I need a smaller example before I can use type and truthiness evidence in a record. |

### My next support move

My issue is mostly: setup / Python syntax / tracing / type conversion / truthiness / confidence / time.
The first value where I got uncertain was: _____
My next small action is: ask a partner / ask instructor / rebuild the type table / rerun one expression / try the recovery version.

*Workbooklet target: I can classify Python values, cast text safely, interpret truthiness, and prepare typed evidence before threshold decisions.*

# Bridge Day 3 VIP Evidence Handoff

STUDENT COPY • Contract 07a04826c975 • Accessible v5 successor

## Why engineers use this page

Revisit the same calculation notebook through type evidence: preserve raw input, convert numeric text, calculate, and format output before independent work.

**Decision boundary:** Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

## Construct readiness before independent work

**New authoring tools:** numeric literal, arithmetic expression, floor division, modulo, exponentiation, input call, int conversion, float conversion, f string, import statement, math library call
**Carried authoring tools:** string literal, assignment, print call, sequential execution, exact output
**Supplied-code exposure only:** none
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

## Notebook cells and task constructs

| Activity | Work | Stable cells | Required authoring constructs |
|---|---|---|---|
| **18.3** | Minutes Conversion | `18-3-work / 18-3-transfer / 18-3-cold` | arithmetic expression |
| **18.4** | Square Root Check | `18-4-work / 18-4-transfer / 18-4-cold` | f string |
| **18.5** | Kit Count Plan | `18-5-work / 18-5-transfer / 18-5-cold` | profile-level contract |
| **18.6** | Rectangle Diagonal | `18-6-work / 18-6-transfer / 18-6-cold` | f string |
| **18.7** | Grid Path Distance | `18-7-work / 18-7-transfer / 18-7-cold` | f string, input call, int conversion |

## Readiness evidence

1. Write one example showing the raw text returned by input, the converted numeric value, and the calculation that becomes legal after conversion.

2. For one Studio problem, name the required numeric type, output precision, and a test that would expose a missing or incorrect conversion.

## Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

# Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

| Evidence field | Record |
|---|---|
| **Prediction** | |
| **Observed Result** | |
| **Revision Reason** | |
| **Engineering Meaning** | |
| **Units Or Not Applicable** | |
| **Assumption** | |
| **Model Limit** | |
| **Next Test** | |
| **Help Trigger** | |
| **Minutes Used** | |

## Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.